

1. Matrix Operations (Phép toán ma trận)

Bản gốc (dịch):

Các phép cộng ma trận rất đơn giản:

- $A + B = B + A$
- $(A + B) + C = A + (B + C)$
- $c(A + B) = cA + cB$
- $(c + d)A = cA + dA$
- $c(dA) = (cd)A$
- $A + 0 = A$ (ma trận 0)
- $A + (-A) = 0$

Các tính chất này hiển nhiên nên không cần chứng minh.

Tính chất nhân ma trận:

- $A(BC) = (AB)C$
- $c(AB) = (cA)B = A(cB)$
- $A(B + C) = AB + AC$
- $(B + C)A = BA + CA$

⚠ Lưu ý: Có hai loại nhân:

- Hadamard (nhân từng phần tử): $\odot$
- Nhân ma trận: AB

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])
```

```
A*B # Hadamard
```

```
array([[ 5, 12],  
       [21, 32]])
```

```
A@B # Matrix multiplication
```

```
array([[19, 22],  
       [43, 50]])
```

2. Minh họa phép nhân ma trận

Giải thích:

Quy tắc nhân ma trận được thực hiện bằng cách lấy tích vô hướng giữa hàng và cột.

```
np.sum(A[0,:]*B[:,0])
np.sum(A[1,:]*B[:,0])
np.sum(A[0,:]*B[:,1])
np.sum(A[1,:]*B[:,1])
```

```
19
43
22
50
```

3. SymPy – Cộng ma trận

Giải thích:

Ta định nghĩa các ký hiệu để tính toán đại số chính xác (symbolic computation). Đây là công cụ hữu ích để học đại số tuyến tính.

```
A = sy.Matrix([[a, b, c], [d, e, f]])
A + A
A - A
```

4. Nhân ma trận với SymPy

Giải thích:

Dùng ký hiệu để hiểu rõ quy tắc nhân.

```
A = sy.Matrix([[a, b, c], [d, e, f]])
B = sy.Matrix([[g, h, i], [j, k, l], [m, n, o]])
A*B = A*B
```

5. Tính không giao hoán (Commutability)

Giải thích:

Nhân ma trận không giao hoán:

$$AB \neq BA$$

Ví dụ:

```
A = sy.Matrix([[3, 4], [7, 8]])  
B = sy.Matrix([[5, 3], [2, 1]])  
A*B  
B*A
```

6. Tìm điều kiện để $AB = BA$

Giải thích:

Ta thiết lập hệ phương trình từ:

$$AB - BA = 0$$

Sau đó giải bằng phương pháp Gauss-Jordan.

```
solution = sp.solve(equations, (e, f, g, h))
```

Kết quả:

```
e: h + g*(a - d)/c  
f: b*g/c
```

Dạng tham số:

```
e = c2 + (c1(a - d))/c  
f = (b*c1)/c  
g = c1  
h = c2
```

7. Chuyển vị ma trận (Transpose)

Giải thích:

Chuyển vị là đổi hàng thành cột.

Tính chất:

- $(A^T)^T = A$
- $(A + B)^T = A^T + B^T$
- $(cA)^T = cA^T$
- $(AB)^T = B^T A^T$

```
AB_t == B_t_A_t
```

True

8. Ma trận đơn vị (Identity Matrix)

Giải thích:

Ma trận đơn vị có 1 trên đường chéo chính.

$$AI = IA = A$$

```
(A@I == I@A).all()
```

True

9. Ma trận sơ cấp (Elementary Matrix)

Giải thích:

Ma trận sơ cấp được tạo từ phép biến đổi dòng.

Ví dụ: đổi dòng 1 và 2.

```
E*A
```

 Thực hiện đúng phép đổi dòng.

10. Ma trận nghịch đảo (Inverse Matrix)

Giải thích:

Nếu:

$$AB = BA = I$$

thì:

$$B = A^{-1}$$

```
Ainv = np.linalg.inv(A)
A@Ainv
```

$\approx$ Identity matrix

11. Gauss-Jordan tìm nghịch đảo

Giải thích:

- Tạo ma trận mở rộng $[A \mid I]$
- Biến đổi về dạng rút gọn
- Thu được $[I \mid A^{-1}]$

12. Điều kiện tồn tại nghịch đảo

Giải thích:

Ma trận khả nghịch khi:

$$\det(A) \neq 0$$

Ví dụ:

```
sy.solvers.solve(-6*lamb-465, lamb)
```

 Điều kiện:

$$\lambda \neq -155/2$$



13. Tính chất của ma trận nghịch đảo

Giải thích:

1. $(AB)^{-1} = B^{-1}A^{-1}$
 2. $(A^T)^{-1} = (A^{-1})^T$
 3. Nếu $AB = BA$ thì tính chất đối xứng được bảo toàn
-



Tổng kết

Bạn đang làm việc với một tài liệu rất tốt về:

- Đại số tuyến tính
- Minh họa bằng Python
- Kết hợp NumPy + SymPy